

FLASH-MAS – A Flexible and Lightweight Agent Shell

July 22, 2026

- Hello World
- Configuration
- Messaging

FLASH-MAS – A Flexible and Lightweight Agent Shell

overview

Hello World – A simple Java agent

- An agent must extend the `flash.core.agent.Agent` interface, which extends the `Entity` interface.
- To create an agent more easily one can extend the `flash.core.agent.BaseAgent` class and override the `start()` method.

```
import net.xqhs.flash.core.agent.BaseAgent;

public class HelloWorldAgent extends BaseAgent {
    public boolean start() {
        if (!super.start()) return false;
        li("Hello-World");
        stop();
        return true;
    }
}
```

This code snippet is based on `example.helloWorld.HelloWorldAgent` in the `src-examples` source folder.

Hello World – A simple Java agent

Main startup options summary:

- via command line arguments:

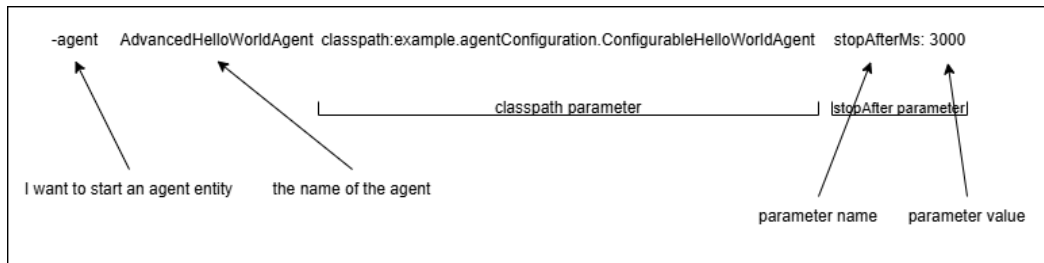
```
java -cp "bin:lib-all/*" quick.Boot -agent HelloWorldAgent  
classpath:example.helloWorld.HelloWorldAgent
```

- via XML deployment file:

```
java -cp "bin:lib-all/*" quick.Boot  
src-examples/example/helloWorld/deployment.xml
```

Sending parameters via CLI

- Configuration needed to start the deployment via command line arguments: the name of the agent, classpath and stopAfter parameters.



Sending parameters via XML

- Configuration needed to start the deployment in an XML deployment file: the name of the agent, classpath and stopAfter parameters.

```
<deployment>
  <agent classpath="example.agentConfiguration.ConfigurableHelloWorldAgent" name="
    HelloWorldAgent">
    <parameter name="stopAfterMs" value="3000"/>
  </agent>
</deployment>
```

This code snippet is based on `example.agentConfiguration.deployment` in the `src-examples` source folder.

Receiving parameters by extending the `EntityCore` class

- The implementation uses the `getConfiguration()` method for extracting `stopAfter` parameter.

```
public class ConfigurableHelloWorldAgent extends BaseAgent {  
  
    public boolean start() {  
        if(!super.start()) return false;  
  
        long stopAfter = getConfiguration().get(STOP_AFTER_PARAMETER_NAME).orElse(  
            DEFAULT_STOP_AFTER);  
  
        Timer t = new Timer();  
        t.schedule(new TimerTask() {...}, stopAfter);  
        return true;  
    }  
}
```

This code snippet is based on `example.agentConfiguration.ConfigurableHelloWorldAgent` in the `src-examples` source folder.

Receiving parameters by overriding the `configure()` method

- The implementation overrides the `configure()` method and extracts the `stopAfter` parameter. The `configure()` method of an entity is called immediately after its construction.

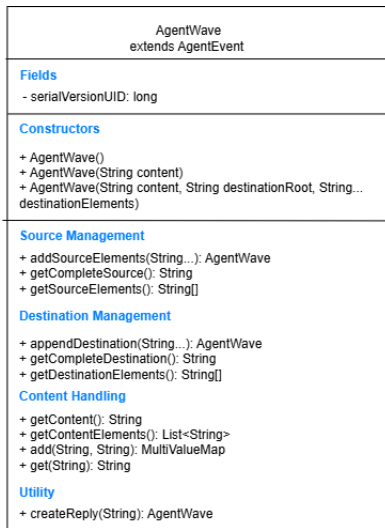
```
public class ConfigurableHelloWorldAgent2 extends BaseAgent {  
  
    @Override  
    public boolean configure(MultiTreeMap configuration) {  
        if (configuration.isSet(STOP_AFTER_PARAMETER_NAME))  
            stopAfter = configuration.getFirstValue(STOP_AFTER_PARAMETER_NAME);  
        else  
            stopAfter = DEFAULT_STOP_AFTER;  
        return true;  
    }  
}
```

This code snippet is based on `example.agentConfiguration.ConfigurableHelloWorldAgent2` in the `src-examples` source folder.

- In FLASH-MAS, an interaction between entities is performed via a *wave*, implemented in the class `AgentWave`.
- A wave has a *source*, a *destination*, and *content*.
- Waves are transmitted between entities either directly (via method calling) or by means of a communication infrastructure, which can transmit waves inside the same node or between different nodes.
- The source and destination are *endpoints*.
- A *message* is a wave sent between two agents, via a communication infrastructure.
- A *Pylon* is an *embodiment* of a communication infrastructure on a node.
- Interaction with a communication infrastructure is done directly (via proxy to the pylon) or via a `messagingShard`, which may be custom for the pylon implementation.

Messages between entities

- UML class diagram of AgentWave in which the most important methods were included



Messages between entities

Scenario for this example

- Two instances of the same agent class, `AgentPingPongPlain`, communicate back and forth by sending and receiving waves through a pylon, which is accessed by entities via a proxy (the `WaveMessagingPylonProxy` class).
- One instance is configured to act as the "ping" initiator (it has a `sendTo` parameter pointing to the other agent's name), while the other implicitly becomes the "pong" responder (it has no `sendTo` parameter)

Messages between entities

Initialization

- The agent is placed, using information from the deployment configuration, in the context of the pylon. The pylon is visible by the agent via a proxy.
- When added in the context of the pylon, the agent *registers* to the pylon, the communication infrastructure knows the node where the agent is located and the name of the agent, so it can receive messages.
- Agent must provide a proxy to the pylon, as a WaveReceiver instance.

@Override

```
public boolean addContext(EntityProxy<Pylon> context) {  
    pylonProxy = (WaveMessagingPylonProxy) context;  
    return pylonProxy.register(getName(), new WaveReceiver() {  
        @Override  
        public void receive(AgentWave wave) {  
            processEvent(wave);  
        }  
    })  
}
```

This code snippet is based on `testing.AgentPingPongPlain` in the `src` source folder.

Sending messages

- The agent sends waves periodically using a timer.
- When the agent starts and it has a list of destinations, it schedules a task that sends a message to each destination every second.

```
protected void sendPing() {  
    AgentWave wave = new AgentWave("ping-no-" + tick, otherAgent, "pong");  
    pylonProxy.send(wave);  
}
```

This code snippet is based on `testing.AgentPingPongPlain` in the `src` source folder.

Receiving messages

- When an agent is registered to the pylon, it provides a receiver implementation that is triggered on incoming waves.

```
protected boolean processEvent(AgentWave wave) {  
    String replyContent = wave.getContent() + "-reply";  
    return send(wave.createReply(replyContent));  
}
```

This code snippet is based on `testing.AgentPingPongPlain` in the `src` source folder.

Scenario

- The agent delegates all messaging to a `MessagingShard` rather than using the `WaveMessagingPylonProxy` directly.

Instantiating the Messaging shard

- The shard is instantiated in the `register()` method using the `instantiateRecommendedShard()` method from the `AgentShardCore` class.
- After instantiation, the shard is bound to the context, which gives it access to the messaging system through the proxy.

```
msgShard = AgentShardCore.instantiateRecommendedShard(MESSAGING,
    (PylonProxy) context, null, new BaseAgentProxy() {
        @Override
        public boolean postAgentEvent(AgentEvent event) {
            return processEvent(event)
        }
    });
msgShard.addGeneralContext(context);
```

This code snippet is based on `testing.AgentPingPong` in the `src` source folder.

Messages between entities

Signaling start event

- When the agent starts, it tells the shard by sending it an `AGENT_START` event.

```
@Override
public boolean start() {
    msgShard.signalAgentEvent(new AgentEvent(AgentEventType.AGENT_START));
}
```

This code snippet is based on `testing.AgentPingPong` in the `src` source folder.

Sending messages

- The `send()` method of the agent now simply delegates to the messaging shard.
- Internally, the shard adds the agent's address to the wave if it is missing, and then forwards the message through either a `wavePylon` or a `classicPylon`, depending on what it has.

```
msgShard.sendMessage(wave);
```

This code snippet is based on `testing.AgentPingPong` in the `src` source folder.

Messages between entities

Receiving messages

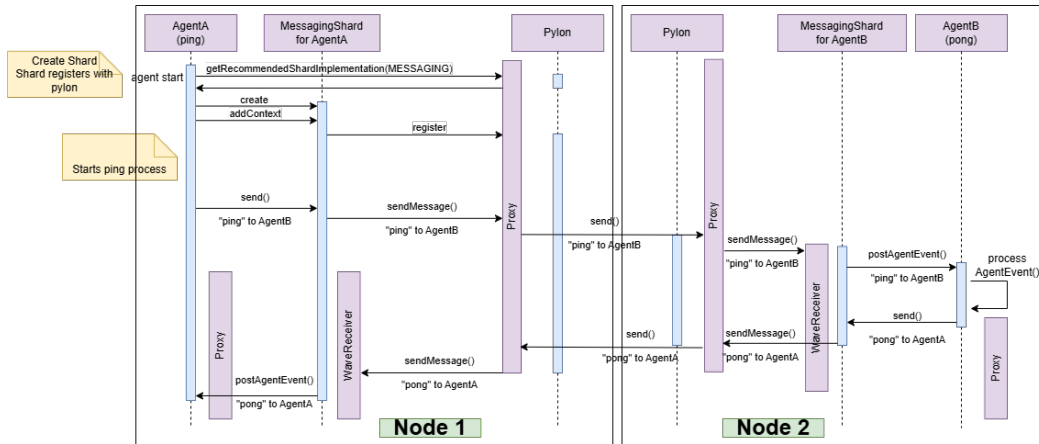
- When a message arrives for the agent, the shard uses the proxy provided during registration.
- The proxy forwards the wave to the agent's `processEvent()` method, presented in the first scenario.

```
public boolean postAgentEvent(AgentEvent event) {  
    return processEvent(event)  
}
```

This code snippet is based on `testing.AgentPingPong` in the `src` source folder.

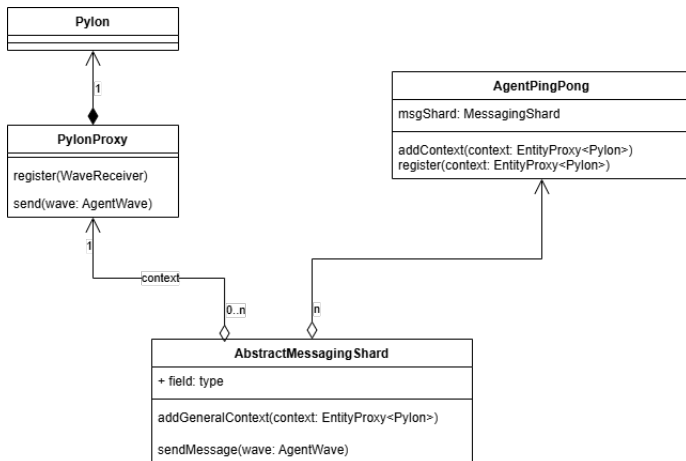
Messages between entities

- Sequence diagram containing the agents, the pylon, and the shards



Messages between entities

- UML class diagram for the agent - shard - pylon - pylon proxy - shard container structuring, especially from a context perspective



Messages between entities

- To start the deployment, one can run the `Boot` class from the `simplePingPong` subpackage under `src-examples/example`.
- In this file, one can select the desired interaction with the communication infrastructure, either directly using `AgentPingPongPlain`, or via a `messagingShard` by means of `AgentPingPong`.
- Regardless of the option, the node, the names of the agents involved, the configuration of the `sendTo` parameter with the names of the agents that will receive messages, as well as the number of pings sent must be provided.